

Reviewer Pack — Asymptotic Dimension and Tseitin Resolution Length

Entry 2f7cf0867b85 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 08:47:42 UTC

Asymptotic Dimension and Tseitin Resolution Length

Entry ID: 2f7cf0867b85 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 08:47:42 UTC

1. Conjecture

Field A (mathematical branch): Geometric Group Theory **Field B** (complexity object): Resolution Length of Tseitin Formulas

Statement:

For a graph G with asymptotic dimension d , the resolution length of the Tseitin formula on G is at least $2^{\Omega(d)}$.

Rationale (proposer’s reasoning):

Asymptotic dimension captures the large-scale geometric complexity of the graph, which may directly influence the length of resolution proofs required to refute the Tseitin formula.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9deda73b591d6058

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_graph(n):
        graph = {i: [] for i in range(n)}
        edges = set()
        while len(edges) < n * (n - 1) // 2:
            u, v = random.sample(range(n), 2)
            if u != v and (u, v) not in edges and (v, u) not in edges:
                graph[u].append(v)
                graph[v].append(u)
                edges.add((u, v))
        return graph

    def tseitin_formula(graph):
        n = len(graph)
        literals = [f"x{i}" for i in range(n)]
        clauses = []

        # Base case
        for i in range(n):
            clauses.append([literals[2*i], literals[2*i+1]])

        # Implication rules
        for u in range(n):
            for v in graph[u]:
                clauses.append([-literals[2*u], literals[2*v]])
                clauses.append([-literals[2*u+1], -literals[2*v+1]])
                clauses.append([literals[2*u], literals[2*v+1]])
                clauses.append([literals[2*u+1], literals[2*v]])

        return literals, clauses

    def resolution_length(clauses):
        n = len(clauses)
        unit_clauses = {c[0] for c in clauses if len(c) == 1}
        resolvents = set()

        while True:
            new_resolvents = set()
            for i in range(n):
```

```

        for j in range(i+1, n):
            if -clauses[i][0] in clauses[j]:
                new_resolvent = [l for l in clauses[i] if l != -clauses[i][0]] + [l for l
                new_resolvents.add(tuple(sorted(new_resolvent)))
            elif -clauses[j][0] in clauses[i]:
                new_resolvent = [l for l in clauses[i] if l != -clauses[j][0]] + [l for l
                new_resolvents.add(tuple(sorted(new_resolvent)))
        if not new_resolvents:
            break
        resolvents.update(new_resolvents)
        unit_clauses.update({c[0] for c in new_resolvents if len(c) == 1})

    return len(resolvents)

def asymptotic_dimension(graph):
    n = len(graph)
    for d in range(1, n+1):
        if all(len(set(graph[v] for v in graph if u not in graph[v])) >= d-1 for u in graph):
            return d
    return n

n_values = [5, 10, 15, 20, 30, 40]
total_length = 0
instances_tested = 0

for n in n_values:
    for _ in range(5): # Ensure at least 30 instances per seed
        graph = generate_random_graph(n)
        d = asymptotic_dimension(graph)
        literals, clauses = tseitin_formula(graph)
        length = resolution_length(clauses)
        total_length += length
        instances_tested += 1

mean_length = total_length / instances_tested
conjecture_holds = mean_length >= 2 ** (0.5 * d) for d in range(1, n+1)

return {
    "metric_name": "resolution_length",
    "metric_value": mean_length,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_length = sum(r["metric_value"] for r in results) / len(results)

```

```

support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_length} std=0 support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_98637aa2.py", line 117, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_98637aa2.py", line 96, in run_trial
    literals, clauses = tseitin_formula(graph)
                        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_98637aa2.py", line 41, in tseitin_formula
    clauses.append([-literals[2*i], literals[2*j]])
                    ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with IndexError before producing valid results | next: Fix the Tseitin formula implementation to handle graph vertex indexing properly

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	100245
2	propose	ollama_remote	qwen3:8b	0	0	116228
3	propose	ollama_remote	qwen3:8b	0	0	138188
4	propose	ollama_remote	qwen3:8b	0	0	153503
5	preregistration	ollama_remote	qwen3:8b	0	0	31690

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	novelty	ollama_remote	qwen3:8b	0	0	27414
7	novelty	ollama_remote	qwen3:8b	0	0	23552
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14815
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14730
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12076
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12980
12	judge	ollama_remote	qwen3:8b	0	0	21274

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 666695 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/2f7cf0867b85.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2f7cf0867b85.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/2f7cf0867b85.tar.gz` (if generated)